



interfaceNode - *Manual*

Version 1.0.5

Hardware Revision: B.0.x

The Atkin Engineering **interfaceNode** is a Wireless RS-485 bridge that provides bridging connections between RS-485, USB, TCP sockets, and WebSockets for various control devices.

front

rear



Contents

- [Contents](#)
- [1. Connections](#)
- [2. Block diagram](#)
- [3. Operation flow](#)
- [4. Default mode](#)
 - [Default password](#)
- [5. System button](#)
- [6. Config mode](#)
 - [First connection and trusting the device](#)
 - [Device](#)
 - [Network](#)
 - [Protocol](#)
 - [Bridge](#)
 - [Security](#)
 - [Encryption \(TLS\)](#)
 - [Device certificate](#)
 - [Saving and maintenance](#)
 - [USB drive](#)
- [7. Operation mode](#)
 - [Access point](#)
 - [Station](#)
- [8. Sockets](#)
 - [TCP bridge security](#)
 - [WebSocket bridge](#)
 - [WebSocket DyNet filtering](#)
 - [DyNet discovery beacon](#)
- [9. Serial](#)
- [10. Led status](#)
 - [RS485 LED](#)
 - [WiFi LED](#)
- [11. System](#)
 - [Factory reset](#)
 - [System log](#)
 - [Security events](#)
 - [Watchdog](#)
- [12. Firmware](#)
 - [New releases and security fixes](#)
- [13. Security information](#)

1. Connections

Front

- USB Type-C, USB 2.0 full-speed port [[^]usbpower]
- 1x Status LED
- 1x System button

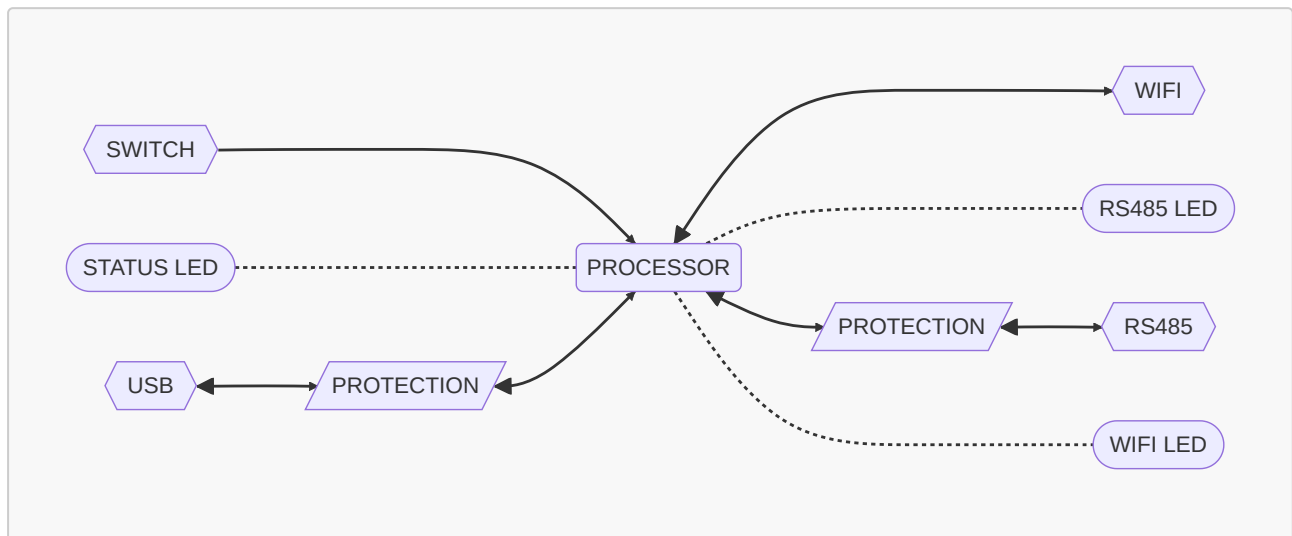
Rear

- 2x RS-485 (parallel RJ12 6P6 ports) supplies power & data [[^]rs485power]
- 2x RGB status LEDs
- Full / Half duplex (hardware jumper - see [[^]jumpers])
- Termination **None** / **120ohm** (hardware jumper - see [[^]jumpers])

RS-485 pinout (RJ12 6P6)

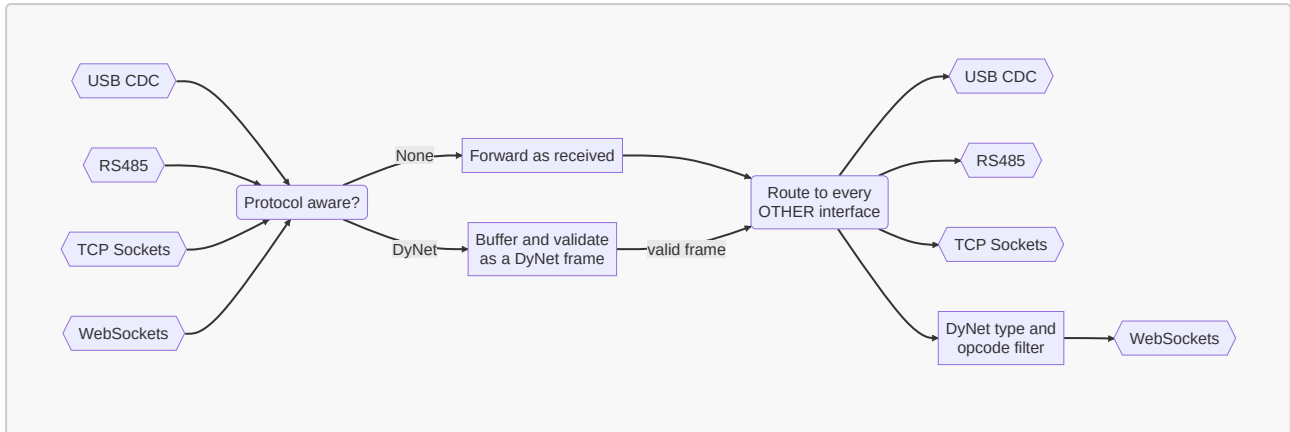
1. Ground
2. Ground
3. Data +
4. Data -
5. 12/24 VDC
6. 12/24 VDC

2. Block diagram



3. Operation flow

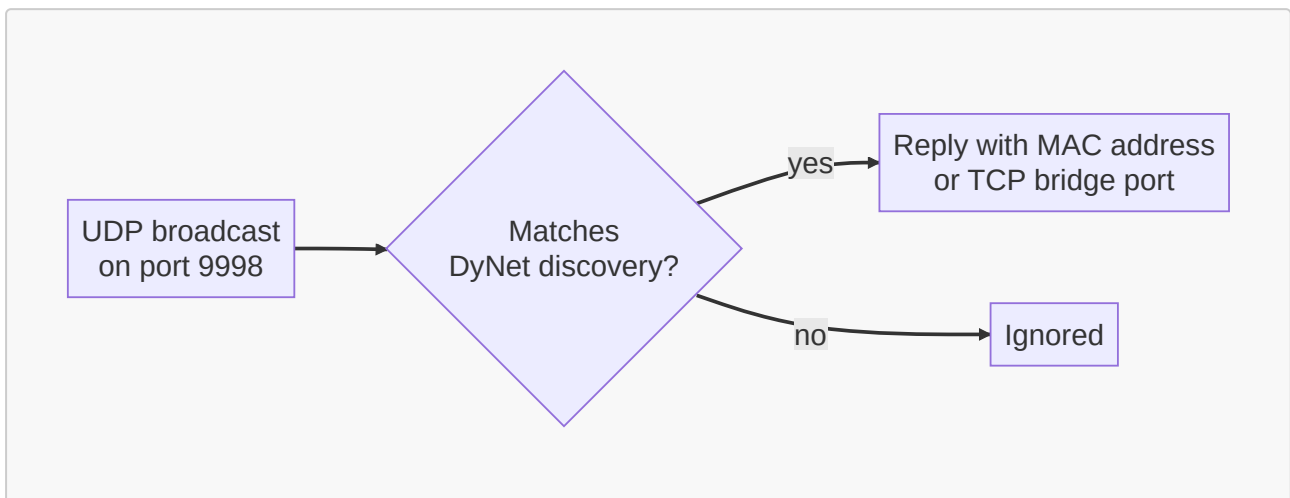
Data arriving on USB, RS485, a TCP socket or a WebSocket is handled the same way, whichever interface it came in on:



A frame is never sent back out of the interface it arrived on. With **Protocol aware** set to **DyNet**, incomplete data is held until the rest of the frame arrives, and data that fails its checksum is discarded while the stream re-synchronises on the next valid frame. With it set to **None**, data is passed straight through with no validation.

Only WebSocket delivery is filtered - see [WebSocket DyNet filtering](#). USB, RS485 and TCP always receive every frame.

The DyNet discovery beacon (see [Sockets](#)) is a separate process. It does not pass through the flow above, and nothing it receives or sends is routed to any bridge interface:



4. Default mode

After a factory reset, **interfaceNode** starts in **configuration mode**, ready to be set up. Its settings are the defaults below; leaving configuration mode (a short press of the **system button**) starts the bridge in **Access point** mode with:

- SSID: **interfaceNodeXXYY**, where **XXYY** is the device's serial number (the last two bytes of its MAC address, in hex - see [Device](#))
- Password: the device's own default password - see [Default password](#)
- Security: **WPA2/WPA3 PSK**
- IP address: **192.168.4.1**, with a DHCP server for connected devices
- **Protocol aware**: **DyNet**
- TCP port: **50000**, TLS on, login required (see [TCP bridge security](#))
- Web interface over HTTPS; WebSocket bridge over TLS (**wss://**), login required (see [Security](#))
- RS485 baud rate: **9600**

- WebSocket DyNet filtering: DyNet 1 forwarded with opcodes [0-4](#), [10-13](#), [96](#), [98](#), [101](#), [113-115](#), [128-131](#); DyNet 2 not forwarded (opcode list [1](#), [16](#), [33](#), [35](#) once forwarding is on) - see [WebSocket DyNet filtering](#)

Note: a firmware update keeps your settings - see [Firmware](#).

Default password

Every **interfaceNode** has its own default password, used for the default access point, for [configuration mode](#), and with the username [admin](#) for the web interface, WebSocket and TCP logins (see [Security](#)). There is no shared factory password.

- Three groups of four characters separated by dashes, [XXXX-XXXX-XXXX](#) - for example [7KQ2-9MXA-3RTC](#). The characters are digits and capital letters, without [I](#), [L](#), [0](#) and [U](#), so none can be mistaken for another ([0](#) is always zero, [1](#) always one).
- Type it exactly as printed, **including the dashes** - for the WiFi network as well as the logins.
- Set during manufacturing from a random key created inside the device's chip and locked there, so it can't be read out or worked out from the MAC address, serial number, network name or firmware.
- Printed on the device label and on the password card supplied with the device.
- Not changed by a factory reset - the device always returns to the same default password.
- If both the label and the card have been lost, contact Atkin Engineering with the device's MAC address or serial number (shown under [Device](#) in the web interface, in the network name, or on the label). You'll be asked to show you own the device, for example with a photo of the label or the invoice.

Note: changing the access point password in the web interface replaces the default password for operating mode only. Configuration mode always uses the default password, so the device can still be reached for setup.

5. System button

The system button supports two gestures:

- [Short press](#) Switches between operating mode and configuration mode: the device reboots into configuration mode if it is operating, or back into operating mode if it is already in configuration mode. See [System](#) for what configuration mode means.
- [2 second hold](#) Full factory reset - erases all user data and reboots into [configuration mode](#) with the [default settings](#). See [Factory reset](#).

Note: the button is not active immediately at power-on - it responds once the rest of the system (network, bridge or web interface, beacon) has started. It also stops responding once a reboot has been triggered by any means (the button itself, or an action in the web interface); a press during the short delay before the restart is ignored.

6. Config mode

Configuration mode runs the web interface used to view and change the device's settings. See [System](#) for how it relates to normal bridge operation, and [Default mode](#) / [System button](#) for how to enter and leave it.

In configuration mode the device runs its own access point, `interfaceNodeXXYY`, secured with its [default password](#). Join that network, then open the device in a web browser at its IP address (`https://192.168.4.1` in [default mode](#)), or by its mDNS hostname, e.g. `https://interfacenode3080.local` for serial number 3080. mDNS works on most networks and operating systems without any setup. Typing the address without `https://` also works: plain `http://` (port 80) redirects to `https://`.

The web interface is served over HTTPS (TLS) with a certificate the device issues itself (see [Device certificate](#)). Until the computer trusts the device, the browser shows a warning ("Your connection is not private", "Warning: Potential Security Risk Ahead" or similar): choose to continue (for example [Advanced](#) > [Proceed](#)), then download the device's CA certificate and install it, once per computer - see [First connection and trusting the device](#). From then on there are no warnings in browsers or other software, even when the device's IP address changes. With HTTPS turned off (see [Security](#)) the interface is plain `http://` on port 80.

The web interface asks for a username and password first. The default is `admin` with the device's [default password](#). After 5 wrong passwords in a row, logging in is refused for 30 seconds, doubling with each further wrong password up to 10 minutes. A login lasts until [Log out](#), 15 minutes without use, or the device restarting.

The page is divided into the sections below. Only the settings that apply are shown: the access point or station settings for the selected WiFi mode, and the DyNet filters only when [DyNet](#) is selected. Hardware-only settings (RS485 termination and duplex - see [Connections](#)) are not part of the web interface.

First connection and trusting the device

Do this once on each computer that will use the device - the configuration computer, and any computer running software that connects to the WebSocket or TCP bridge with TLS. Afterwards, browsers and other software connect without certificate warnings, and keep doing so when the device's IP address changes (for example when it moves from its own access point to your network, or gets a new address by DHCP).

1. **Enter configuration mode.** A new or factory-reset device starts in it; otherwise give the [system button](#) a short press.
2. **Join the device's network.** Connect the computer's WiFi to `interfaceNodeXXYY` (XXYY is on the device label), using the device's [default password](#) from the label. If you've changed the WiFi settings, join that network instead.
3. **Open the web interface.** Go to `https://192.168.4.1` (or `https://interfacenodeXXYY.local`). If you've set a custom access point address, use that.
4. **Continue past the warning, this one time.** The computer doesn't trust the device yet, so the browser warns ("Your connection is not private" or "Warning: Potential Security Risk Ahead"). As you are connected directly to the device's own network, continue: Chrome and Edge [Advanced](#) > [Proceed to 192.168.4.1 \(unsafe\)](#); Firefox [Advanced](#) > [Accept the Risk and Continue](#); Safari [Show Details](#) > [visit this website](#).
5. **Log in** with `admin` and the default password (or your own login).
6. **Download the device CA certificate.** Under [Security](#) > [Device certificate](#), select [Download device CA certificate](#). The file is called `interfacenodeXXYY-ca.crt`.
7. **Check it (recommended).** Compare the file's SHA-256 fingerprint with [CA SHA-256 fingerprint](#) on the page:
 - Windows: `certutil -dump interfacenodeXXYY-ca.crt` and look at `Cert Hash(sha256)`
 - macOS: open the file; the fingerprints are at the bottom of the details
 - Linux: `openssl x509 -in interfacenodeXXYY-ca.crt -noout -fingerprint -sha256`

They must match (ignore spaces, colons and letter case). If they don't, don't install it.

8. **Install it as a trusted root certificate**, following the steps for your system under [Device certificate](#). On Windows choose [Local Machine](#) and the [Trusted Root Certification Authorities](#) store - not "Automatically select", which puts it in the wrong store. On a Mac set it to [Always Trust](#).
9. **Restart the browser** (close every window), and restart any other program that connects to the device, so they pick up the new certificate.
10. **Check it worked**. Open <https://192.168.4.1> (or <https://interfacenodeXXYY.local>) again: there should be no warning, and the address bar shows the usual padlock.

Then set up the device (WiFi, logins and so on) and [Save and reboot](#). When it joins your network in station mode, connect to it at its new address or at <https://interfacenodeXXYY.local> - the device re-issues its certificate for the new address by itself, so the computer still trusts it.

When it still warns:

What you see	Why, and what to do
Warning only at some addresses	Connect using a name or address listed under Names in Device certificate . Public (internet) IP addresses can't be included - use the device's name
Warning in Firefox, or "certificate verify failed" in Java, Python or Node.js programs	They keep their own list of trusted certificates: import the file there too (see Device certificate)
Warning after a factory reset or Make new device certificates	The device has a new CA: download and install its certificate again. You can delete the old one (interfacenodeXXYY device CA) from the computer's certificate store
Works in the browser, not in a program running as a Windows service	The certificate was installed for the current user only: install it again choosing Local Machine
Still warns everywhere	Check it was installed under Trusted Root Certification Authorities (Windows) or set to Always Trust (Mac), then restart the browser

Device

Read-only details: status, serial number, hostname, MAC address, firmware version, build date, SDK version, the running firmware partition and the device time.

The device has no battery-backed clock and doesn't use the internet, so each time you log in, the page sets the device's date, time and time zone from your computer. The device time is shown here and added to each [system log](#) entry. It keeps running through a restart, but is lost when power is removed until the next login; until then the log shows only the time since start-up. A correction of a minute or more is recorded as a [security event](#).

Network

Setting	Values
WiFi mode	Access point / Station / No WiFi - see Operation mode
Access point Network name	Text
Access point Password	Text, 8+ characters
Access point Channel	1-13
Access point Security	WPA PSK , WPA2 PSK , WPA/WPA2 PSK , WPA3 PSK , WPA2/WPA3 PSK
Access point Device IP address	Default (192.168.4.1) / Custom (IP address, network mask, gateway, DNS)
Station Network name	Text
Station Security	Open , WEP , WPA PSK , WPA2 PSK , WPA/WPA2 PSK , WPA3 PSK , WPA2/WPA3 PSK , WPA2 Enterprise , WPA3 Enterprise

Setting	Values
Station Password	Text, 8+ characters
Station Enterprise sign-in	Username, identity, server certificate validation and certificate upload (Enterprise security only)
Station Device IP address	Automatic (DHCP) (default) / Static (IP address, network mask, gateway, DNS)

Protocol

Setting	Values
Protocol awareness	None / DyNet - see Operation flow
DyNet 1 opcodes	List of opcodes sent to WebSocket clients (default 0-4, 10-13, 96, 98, 101, 113-115, 128-131) - see WebSocket DyNet filtering
DyNet 2 opcodes	As above, for DyNet 2 (default 1, 16, 33, 35)

Bridge

Setting	Values
TCP port	49152-65535 - see Sockets
RS-485 baud rate	9600 / 19200 / 38400 / 57600 / 115200 - see Serial

Security

A coloured border around the whole page, and a line at the top of it, show how well the services are protected, so a weakened setup is noticed straight away:

Border	Meaning
Green - Secured	Every service (web interface, WebSocket bridge, TCP bridge) has a login and TLS
Amber - Login only	Every service has a login, but at least one has TLS off: its usernames, passwords and data can be read on the network
Red - Not secured	At least one bridge is Open (no login): anyone on the network can use it

The line names the services concerned and links to these settings. It follows the settings on the page as you change them, and says so while changes aren't saved yet.

Setting	Values
Web interface Username	Text, 1-31 characters, no : (default admin)
Web interface New password	Text, 8-63 characters, entered twice. Leave blank to keep the current password
Use TLS for	Web interface, WebSocket bridge, TCP bridge - each on (default) or off. See Encryption (TLS)
WebSocket clients	Login required (default) / Open (no login) - see WebSocket bridge
WebSocket Username	Text, 1-31 characters, no : (default admin)
WebSocket New password	Text, 8-63 characters, entered twice. Leave blank to keep the current password

Setting	Values
TCP clients	Login required (default) / Open (no login) - see TCP bridge security
TCP Username	Text, 1-31 characters, no : (default admin)
TCP New password	Text, 8-63 characters, entered twice. Leave blank to keep the current password
Web pages from other sites	Allow (default) or not - cross-origin requests (CORS) to the web interface and the WebSocket /auth URL. Needed by web apps served from elsewhere, such as dynet_ui . Logins are still required either way
Security event log	Record in the system log on (default) or off - see Security events
Device certificate	Download the device CA certificate to install on computers; shows the certificate in use; upload your own, or make new ones - see Device certificate

Passwords are never shown or sent back by the device - password fields (including the WiFi passwords under [Network](#)) are always empty, and a password only changes when a new one is typed. The device stores only a salted hash of each login password (PBKDF2-HMAC-SHA256), never the password itself.

Note: choosing [Open \(no login\)](#) lets anyone on the network send and receive DyNet traffic over that bridge. Use it only on a network you trust, for clients that can't log in.

Encryption (TLS)

With TLS on (the default for all three), traffic is encrypted with TLS 1.2 (ECDHE key exchange, AES-GCM), so passwords and DyNet traffic can't be read or changed by others on the network:

Service	TLS on	TLS off
Web interface	https://<device>/ (port 443); http:// redirects to it	http://<device>/ (port 80)
WebSocket bridge	wss://<device>/ws , login at https://<device>/auth (port 443)	ws://<device>/ws , http://<device>/auth (port 80)
TCP bridge	TLS on the TCP port	Plain TCP on the TCP port

The WebSocket bridge uses the standard ports, so no port needs entering in a client: 443 with TLS, 80 without. In operating mode nothing else uses them - the web interface only runs in configuration mode, when the WebSocket bridge doesn't. With TLS on, port 80 is closed. Clients must use the matching form: a plain client can't connect to a TLS service, and the reverse. Turning TLS off is a deliberate choice for clients that can't use it, and leaves that traffic - including passwords - readable by anyone on the network. A warning is shown on the page while any service has TLS off. Changes take effect when the settings are saved (the device restarts).

If the web interface can't start HTTPS (for example if no certificate could be made), it falls back to plain HTTP so the device can still be set up, and logs an error. The WebSocket and TCP bridges don't start without a certificate while their TLS is on.

Device certificate

Each **interfaceNode** is its own small certificate authority (CA). On its first start (and after a [factory reset](#)) it makes:

- A **device CA certificate**, [CN=interfacenodeXXYY device CA](#). This is the certificate to install on computers. It stays the same until a factory reset or [Make new device certificates](#). It can only vouch for this device's names ([interfacenodeXXYY](#), [interfacenodeXXYY.local](#)) and for private network addresses (10.x.x.x, 172.16-31.x.x, 192.168.x.x, 169.254.x.x and 100.64-127.x.x), so trusting it can't let anyone impersonate other websites.
- The **device certificate** used for HTTPS, WSS and TLS on the TCP bridge, signed by the device CA. It names [interfacenodeXXYY](#), [interfacenodeXXYY.local](#) and the device's current IP address. When the address changes - a new DHCP address, a new static address, or switching between access point and station - the device issues a new one by itself and uses it straight away. A computer that trusts the device CA accepts it without anything being installed again.

Both use ECDSA P-256 keys, made on the device; the keys never leave it and are kept in encrypted storage. Both are valid until 2049: the device has no clock to renew a short-lived certificate.

Trusting the device (once per computer). Select **Download device CA certificate** (or open https://<device>/tls_cert/ca.crt), then install the file - the full procedure is in [First connection and trusting the device](#):

System	How to install the device CA certificate
Windows (Windows software, Chrome, Edge)	Double-click the file > Install Certificate > Local Machine > Place all certificates in the following store > Trusted Root Certification Authorities > Finish . Or, in a command prompt run as administrator: <code>certutil -addstore -f Root interfacenodeXXYY-ca.crt</code>
macOS (Safari, Chrome, Mac software)	Double-click the file and add it to the System keychain. In Keychain Access open it, expand Trust and set When using this certificate to Always Trust . Or: <code>sudo security add-trusted-cert -d -r trustRoot -k /Library/Keychains/System.keychain interfacenodeXXYY-ca.crt</code>
Linux	<code>sudo cp interfacenodeXXYY-ca.crt /usr/local/share/ca-certificates/ && sudo update-ca-certificates</code>
iPhone, iPad	Open the file and install the profile in Settings, then turn it on in Settings > General > About > Certificate Trust Settings
Android	Settings > Security > Encryption & credentials > Install a certificate > CA certificate
Firefox, Java, Python and Node.js programs	These keep their own list of trusted certificates: import the file into the program (Firefox: Settings > Privacy & Security > View Certificates > Authorities > Import)

Before installing, you can check the file is the device's own: its SHA-256 fingerprint is shown as **CA SHA-256 fingerprint** here and printed in the system log when it is made.

Connect using one of the names or addresses listed under **Names**. A public (internet) IP address can't be included, since the device CA may only vouch for private addresses; use the device's name in that case.

Other actions:

- **Upload certificate and key** Use your own certificate (PEM, optionally with its chain) and its unencrypted private key (PEM), for example one issued by your organisation's certificate authority. Both files must be under 8 KB; the device checks that the key matches the certificate before storing it. ECDSA and RSA keys are accepted. An uploaded certificate is used as it is: the device doesn't add its addresses to it, so it must name every address or name clients use.
- **Make new device certificates** Makes a new device CA and device certificate with new keys, for example if a key may have been exposed. It also replaces an uploaded certificate. Every computer must then install the new CA certificate.

Changes apply to new connections straight away; no restart is needed. A **factory reset** erases the device CA, the device certificate and any uploaded certificate; new ones are made on the next start, and computers must install the new CA certificate.

Saving and maintenance

Action	Effect
Save and reboot	Stores every setting and reboots into operating mode
Reboot	Restarts the device without changing any settings
Factory reset	Erases all user data and reboots with factory defaults - see Factory reset
Install firmware	Updates the firmware - see Firmware

See [System](#) for details.

USB drive

In configuration mode the USB Type-C port works as a USB drive instead of a serial port. Connect it to a PC and a 6 MB drive named **interfaceNode storage** appears, which can be read and written like a USB stick.

- `device_info.txt` Written by the device each time it starts in configuration mode: serial number, MAC address, hostname, mDNS name, WiFi name, firmware version and build, and the running firmware partition. It never contains passwords. Changes made to it are overwritten at the next start.
- Other files are kept until deleted, including across firmware updates. Settings are stored separately and are never visible on the drive.

Note: eject the drive on the PC before saving settings, rebooting or pressing the system button, so files being written are not lost. Do not store passwords or other secrets on the drive - anyone with the device and a USB cable can read it.

7. Operation mode

interfaceNode can operate in three WiFi configurations:

1. **Access point** Creates a WiFi network for other devices to connect to.
2. **Station** Connects to an existing WiFi network as a client.
3. **No WiFi** WiFi is switched off. The bridge runs between USB and RS485 only; the TCP bridge, WebSocket bridge, DyNet discovery beacon and mDNS are not started. Configuration mode still uses its access point, so the device can always be set up.

The **Protocol awareness** setting controls how the bridge handles data:

- **None** Data is forwarded between interfaces as received, with no validation.
- **DyNet** Data is buffered and validated as DyNet 1 or DyNet 2 frames (checksum-checked). Invalid data is discarded and the stream re-synchronises automatically.

Access point

- **Network name** SSID of the access point.
- **Password** Password for the access point.
Note: must be at least 8 characters. A shorter password is not accepted and the device's **default password** is used instead.
- **Channel** WiFi channel (1-13).
- **Security** Security type: **WPA PSK**, **WPA2 PSK**, **WPA/WPA2 PSK**, **WPA3 PSK** or **WPA2/WPA3 PSK**. **WPA2/WPA3 PSK** (the default) is recommended. **Open** and **WEP** are not available for the access point; if either is found in saved settings, the device switches to **WPA2/WPA3 PSK**.

IP address settings:

- **Default** **192.168.4.1**, with the built-in DHCP server.
- **Custom** Set your own IP address, network mask, gateway and DNS server. Leaving gateway or DNS blank offers none to connected devices.

Note: configuration mode always uses the default address, so the device can always be reached for setup even if a custom address is wrong.

Station

- **Network name** SSID of the network to join.
- **Security** Security type of the network being joined: **Open**, **WEP**, **WPA PSK**, **WPA2 PSK**, **WPA/WPA2 PSK**, **WPA3 PSK**, **WPA2/WPA3 PSK**, **WPA2 Enterprise** or **WPA3 Enterprise**.
- **Password** Password of the network being joined. Must be at least 8 characters. For Enterprise networks this is the sign-in password.

If **WPA2 Enterprise** or **WPA3 Enterprise** is selected, an **Enterprise sign-in (EAP)** group appears:

- **Username** EAP username.
- **Identity** EAP identity string.
- **Validate server certificate / Don't validate** Whether to check the RADIUS server's certificate.
- **RADIUS / CA certificate** File picker and **Upload certificate** button. Upload the RADIUS server's CA certificate (PEM format) to enable validation.
Note: the certificate is used from the *next* WiFi connection attempt. If validation is selected but no certificate has been uploaded, the connection proceeds without validation.

PEAP, TTLS and EAP-FAST are supported; the RADIUS server selects the method.

If the device can't join the network, it tries 5 times, waiting 5, 10, 20 and 40 seconds between attempts (about two minutes in all, so a router restart or short power cut is ridden out). If all 5 fail, it records the reason in the **system log** - for example "network not found" or "authentication failed - wrong password?" - and restarts in **configuration mode** with its

own access point, so the settings can be checked. The same happens if an established connection drops and can't be restored. Configuration mode stays on until it is left with the system button or **Save and reboot**.

IP address settings:

- **Automatic (DHCP)** Network-assigned IP address (default).
- **Static** Manually set IP address, network mask, gateway and DNS server.

8. Sockets

interfaceNode runs a TCP bridge server that multiple clients can connect to over the network, in either Access Point or Station mode. None of the network services in this section run in **No WiFi** mode.

- **TCP port** Listening port. Must be between 49152 and 65535.

Every TCP client is bridged the same way as USB and RS485 - see [Operation flow](#). Data received from one TCP client is forwarded to USB, RS485, WebSocket clients and every *other* TCP client, never back to the client that sent it. Up to 5 TCP clients can be connected at once. A further client is disconnected as soon as it connects.

The device also advertises itself by mDNS, so compatible software can find it on the network. Only services that have started are advertised, each under one standard service type that follows its TLS setting, so a client sees the form it can actually connect with:

Service	TLS on	TLS off	TXT records (besides device details)
Web interface (configuration mode)	<code>_https._tcp</code> , port 443	<code>_http._tcp</code> , port 80	-
TCP bridge	<code>_ssl._tcp</code> , TCP port	<code>_socket._tcp</code> , TCP port	<code>tls (1/0)</code> , <code>auth (1/0)</code>
WebSocket bridge	<code>_wss._tcp</code> , port 443	<code>_ws._tcp</code> , port 80	<code>path=/ws</code> , <code>authpath=/auth</code> , <code>tls (1/0)</code> , <code>auth (1/0)</code>

TCP bridge security

With TLS on (the default - see [Encryption \(TLS\)](#)), a client must open a TLS session on the TCP port before anything else. The device presents its [certificate](#); install the device CA certificate on the client computer (or configure it in the client program) so the client can check it.

Unless TCP access is set to **Open** (see [Security](#)), the first thing a client sends (inside TLS, when TLS is on) must be one line:

```
AUTH <username>:<password>
```

ending with a line feed (`\n`, optionally preceded by `\r`). The device answers:

- **OK** - logged in. The connection then carries DyNet exactly as before; any bytes sent after the line in the same packet are treated as DyNet data.
- **ERR login failed** - wrong username or password, or the first line wasn't an **AUTH** line. The device disconnects.
- **ERR locked <seconds>** - after 5 wrong attempts in a row, logins are refused for 30 seconds, doubling up to 10 minutes. The device disconnects.

Each answer ends with `\r\n`. Until it has logged in, a client receives nothing. A client that hasn't finished the TLS handshake and logged in within 10 seconds is disconnected.

The TCP port always accepts the network connection itself, whichever mode is set; the device then drops a client using the wrong form straight away and says why in the [system log](#):

- TLS on, plain client: disconnected as soon as it sends anything that isn't a TLS handshake, or after 3 seconds if it sends nothing.
- TLS off, TLS client: disconnected as soon as its TLS handshake arrives. With **Open** access a client is bridged as soon as it connects (after the TLS handshake, if TLS is on).

Note: most existing DyNet software that connects over plain TCP can't use TLS or send a login line. For those, turn TLS off and choose **Open** for the TCP bridge - only on a network you trust.

WebSocket bridge

interfaceNode also runs a dedicated WebSocket server at `wss://<device address>/ws` on port 443 (`ws://<device address>/ws` on port 80 with TLS off - see [Encryption \(TLS\)](#)), separate from the TCP bridge and from the configuration web interface. It runs in operating mode only. Up to 5 WebSocket clients can connect at once, and each is bridged the same way as USB, RS485 and TCP. While 5 are connected, a further client's connection attempt times out.

Unless WebSocket access is set to **Open** (see [Security](#)), a client must log in, in the same way as the Signify PDDEG-S gateway (but the PDDEG-S uses `/api/v1/dynet/auth`; in `dynet_ui` set the Auth URL to `/auth`):

1. Request a token: `GET https://<device address>/auth` (`http://` with TLS off) with an **Authorization: Basic** header holding the WebSocket username and password. The reply is a one-time token (32 hex characters), valid for 30 seconds. A wrong username or password gives **401**; after 5 wrong attempts in a row requests are refused (**429**) for 30 seconds, doubling up to 10 minutes.
2. Connect to `wss://<device address>/ws` (`ws://` with TLS off) and send the token as the first message, as text.

Note: with TLS on, a browser page (such as `dynet_ui`) can only connect once the browser trusts the device's certificate. Install the device CA certificate on the computer (see [Device certificate](#)), or open `https://<device address>/auth` in the same browser once and continue past the warning. In `dynet_ui` leave **Port** blank (the default: 443 when **Secure** is ticked, 80 when not).

Until it has logged in, a client receives nothing, anything else it sends disconnects it, and it is disconnected after 10 seconds.

A client using the wrong form is refused at once: `ws://` or `http://` while TLS is on fails its TLS handshake, and `wss://` or `https://` while TLS is off is answered **400 Bad Request**. A request for any path other than `/auth` and `/ws` is answered **404**. Each of these is recorded in the [system log](#) with the client's address and the likely cause. With **Open** access a token is not needed; clients configured to fetch one still work. The auth URL allows requests from web pages on other sites (CORS) unless **Web pages from other sites** is turned off under [Security](#).

Data is exchanged as binary WebSocket messages. Clients should send binary messages: the device forwards the bytes of any message it receives, so a text message would put its character codes on the bus rather than the intended bytes.

WebSocket DyNet filtering

With **Protocol awareness** set to **DyNet**, the DyNet frames sent to WebSocket clients can be limited, for example to reduce traffic to a browser-based display. USB, RS485 and TCP are not affected.

Two rules apply to each DyNet type (DyNet 1 and DyNet 2):

1. **Forwarding** - whether that frame type is sent to WebSocket clients at all. By default DyNet 1 is forwarded and DyNet 2 is not. These two switches are not yet in the web interface.
2. **Opcode list** - which opcodes of that type are sent. Only frames whose opcode is listed are forwarded; all others are dropped. The opcode is byte 3 of a DyNet 1 frame and byte 2 of a DyNet 2 frame.

Opcode lists are set in the web interface under **Protocol**. A list is one or more values from **0** to **255**, separated by commas, with a dash for a range:

List	Opcodes sent
0-255	All
1-255	All except opcode 0
1, 5, 20-30	1, 5, and 20 to 30

The defaults are **0-4, 10-13, 96, 98, 101, 113-115, 128-131** for DyNet 1 and **1, 16, 33, 35** for DyNet 2. The web page shows each default under its list, says **Changed from the default** when the list differs (spaces are ignored), and offers **Use default** to put it back. An invalid list is rejected when saving, with a message saying what is wrong; a damaged stored list is replaced with its default.

A firmware update keeps the lists already saved on a device; the new defaults apply after a factory reset, or with **Use default**.

Data that is not a DyNet frame, and all data when **Protocol awareness** is **None**, is always sent to WebSocket clients.

DyNet discovery beacon

When **Protocol awareness** is set to **DyNet** and the device is operating (not in configuration mode), **interfaceNode** listens for DyNet discovery broadcasts on UDP port **9998** and replies with its MAC address and TCP bridge port.

Note: this is a separate listener - it does not go through the bridge, and discovery traffic is never forwarded to or from any bridge interface. See [Operation flow](#).

9. Serial

- **RS-485 baud rate 9600, 19200, 38400, 57600 or 115200.**

RS485 duplex mode (full or half) and termination are set with hardware jumpers - see [Connections](#). RS485 is bridged to USB, TCP and WebSocket clients the same way as any other interface - see [Operation flow](#).

Note: when DyNet is enabled, the USB port answers System Builder's connection check. System Builder sends a fixed DyNet 2 frame when it starts to confirm it is connected to an RS485 network; a plain RS232/RS485 converter returns it because its receiver hears its own transmissions. **interfaceNode** sends that one frame straight back to the PC in the same way. It is not forwarded to RS485, TCP or WebSocket clients, and no other USB data is echoed.

10. Led status

Rebooting (all LEDs, momentarily, immediately before the device restarts):

- **White** Rebooting - shown whatever triggered the restart (saving settings, factory reset, a firmware update, or the system button). After a factory reset the LEDs stay white for up to about a minute while the device erases its data.

Status LED:

- **Green** Starting up - all three LEDs are green until start-up has finished (in station mode, this includes connecting to the WiFi network), then show the mode below.
- **Yellow** Configuration mode - stays until the device is rebooted out of it.
- **Magenta** Operating in Access point mode.
- **Blue** Operating in Station mode.
- **Cyan** Operating with WiFi switched off (**No WiFi**). The WiFi LED stays off.

RS485 LED

- **Slow red pulse** Idle - alternates between bright and dim red every 2 seconds.
- **Fast red flicker** RS485 data activity - returns to idle 0.8 seconds after the last data.

WiFi LED

- **Slow blue pulse** No TCP client connected.
- **Solid dim blue** At least one TCP client connected, idle.
- **Fast blue flicker** TCP data activity.

Note: the WiFi LED shows TCP connections and data only. The WiFi mode is shown on the status LED.

11. System

Important: the web configuration interface is only available in configuration mode. In normal operation (the bridge running), it is not available, and the bridge doesn't run in configuration mode. Switching between the two is a reboot - see [System button](#).

- **Save and reboot** When any setting has been changed, a bar appears at the bottom of the page with this button. It stores every setting and reboots into operating mode. If the network settings changed, reconnect to the device at its new address afterwards.
- **Reboot** Restarts the device without changing any settings. Unsaved changes are lost.
- **Factory reset** Erases all user data and reboots into [configuration mode](#) with the [default settings](#) - see [Factory reset](#).

Each action asks for confirmation before it runs. The browser also warns before leaving the page with unsaved changes.

Factory reset

A factory reset is started by holding the system button for 2 seconds, or with **Factory reset** in the web interface. Both do the same thing: the device restarts, completely erases all user data, then starts in [configuration mode](#) with the [default settings](#): join its access point **interfaceNodeXXYY** with the [default password](#) and open the web interface to set it up.

Erased:

- All settings, including WiFi network names and passwords, Enterprise (EAP) sign-in details and the DyNet filters
- Uploaded RADIUS/CA certificates
- The device CA, the device certificate, any uploaded certificate, and their keys (new ones are made on the next start; install the new device CA certificate on each computer again)
- All files on the [USB drive](#) (it is reformatted the next time configuration mode starts)
- WiFi data stored by the WiFi driver, and any stored crash report

- The [system log](#)

Not changed: the installed firmware and the device's [default password](#).

Note: erasing takes up to about a minute, during which all LEDs stay white. Keep the device powered until it has started again. If power is lost during the erase, repeat the factory reset.

System log

Recent log output is viewable at [/systemlog.html](#), colour-coded by severity (error, warning, info, debug, verbose). Like the rest of the web interface, it is only available in configuration mode.

The log is kept in a fixed-size, 128 KB in-memory buffer. When full, it clears and starts again.

The log survives a reboot triggered by the device itself ([Save and reboot](#), [Reboot](#), or a short press of the system button) and continues where it left off. A factory reset clears it. It is cleared by a power cycle or a firmware update, which are logged as "STARTING COLD BOOT"; any other restart logs "STARTING WARM BOOT" and keeps the existing entries.

Security events

Security events are marked [\[security\]](#) in the system log:

Event	Recorded
Login (web interface, WebSocket, TCP)	Success or failure, the client's address, and for a failed login the username typed
Lockout	The interface and how long logins are locked after repeated failures
Rejected WebSocket clients	An invalid or expired login token, or data sent before logging in
Settings saved	Which groups of settings changed, and any security switch (TLS, login, CORS) turned on or off
Device certificate	A new device CA made, a device certificate issued (with its names and addresses), or a certificate uploaded; a RADIUS/CA certificate uploaded
Clock	The device time corrected by a minute or more from a browser (old and new time)
Firmware update	Started (from which address), rejected, or installed (old → new version)
Firmware changed	On the first start of new firmware: the previous and new version
Factory reset	Requested (button or web interface), and carried out
Start-up	Why the device started: power-on, a restart by the firmware, the watchdog, a crash or a brown-out

Passwords, keys, login tokens and setting values are never recorded.

The log stays on the device: it is kept in memory only and can be read only from [/systemlog.html](#), after logging in, in configuration mode. Nothing is sent anywhere.

To stop recording security events, turn off [Security event log](#) under [Security](#) and save. Turning it off is itself recorded; after that, security events are only written to the debug serial console. Errors and other device messages are still recorded.

Watchdog

The tasks that move data (RS-485 receive and transmit, USB receive and transmit, the TCP bridge and the DyNet beacon) are watched. If one of them stops responding for 10 seconds, the device restarts automatically and records [restarted by the watchdog](#) in the system log.

12. Firmware

- **Firmware file (.bin)** Choose the firmware file.
- **Install firmware** Confirm, then upload the firmware to **interfaceNode**.

A progress bar shows the update as reported by the device itself:

Message	Meaning
Preparing flash...	Making space for the new firmware (a few seconds)
Writing firmware: N%	Firmware received and written so far
Verifying firmware...	Checking the complete firmware
Update complete...	The firmware was accepted and the device is restarting

Do not disconnect or power off the device until the update is complete. If the update fails, the page says so and the device keeps its current firmware. Only one update can run at a time.

Your settings, including WiFi, logins and the DyNet filters, are kept across firmware updates. If the new firmware adds a setting, it starts from its default value (and is recorded in the [system log](#)); check it under the relevant section after updating. Updating from firmware older than v1.3.0 is the exception: settings start from their defaults once, so note them before that update.

If the new firmware fails to start, the device automatically returns to the previous firmware.

Firmware files @ <https://github.com/AtkinEngineering/interfaceNode/releases>

New releases and security fixes

interfaceNode never connects to the internet, so it can't check for or announce new firmware itself. New firmware, including security fixes, is published on the **Releases** tab of the firmware repository: <https://github.com/AtkinEngineering/interfaceNode/releases>. Check it regularly, or use GitHub's **Watch** → **Custom** → **Releases** on that repository to be notified when a new release is published. The installed version is shown under **Device (Version)**.

Version numbers are **major.minor.patch**:

- **Patch** (for example 1.2.3): security fixes and bug fixes only, no new features. Security fixes are released this way as soon as they are ready, without waiting for the next feature release, and are free of charge.
- **Minor** and **major**: new features or changes.

Support period: Security updates are provided free of charge for 7 years from each unit's production date. The end date (month and year) is printed on the product label and box.

To report a security vulnerability, see [SECURITY.md](#) in the firmware repository. Reports are acknowledged within 1 week, assessed within 3 weeks and fixed within 8 weeks - within 3 weeks if the vulnerability is being actively exploited.

Each release lists its changes in the changelog, and security fixes are marked as such. Every release package also includes a software bill of materials ([SBOM.spdx](#)) listing the software components in the firmware, and a report ([SBOM_report.md](#)) of known vulnerabilities checked against them at release time.

13. Security information

This section gives the security information required by the EU Cyber Resilience Act (Regulation (EU) 2024/2847, Annex II).

Manufacturer and contact

- **Manufacturer:** Atkin Engineering, Parcel Locker 82, Lilydale VIC 3140, Australia - www.atkin.engineering
- **EU authorised representative:** named on the product packaging
- **Contact for security vulnerabilities:** temp-pocket-3g@icloud.com. How to report, and how reports are handled: [SECURITY.md](#) in the [firmware repository](#). Reports are acknowledged within 1 week, assessed within 3 weeks and fixed within 8 weeks (3 weeks if actively exploited).
- **EU Declaration of Conformity:** available from Atkin Engineering on request (see the contact above)

Product identification

Product name, model (**interfaceNode**), hardware revision, serial number, production date and the end date of security support are on the device label and box. The firmware version is shown under [Device](#) in the web interface.

Intended use and security environment

interfaceNode bridges RS-485 control networks (such as DyNet lighting control) with USB, TCP and WebSocket clients on a **private local network** inside a building. It is not designed to be reachable from the internet, and it never connects to the internet itself.

Security features: login on every network interface with lockout after repeated failures, TLS encryption, a unique default password, encrypted storage of logins and keys, a security event log and a full factory reset - see [Security](#).

Situations that can lead to significant security risks

- **Making the device reachable from the internet** (port forwarding, or a public or guest network). Keep it on a private network, behind a firewall.
- **Choosing Open (no login)** for the TCP or WebSocket bridge: anyone on the network can then control the RS-485 network. Use it only on a trusted network, for clients that can't log in.
- **Turning off TLS:** passwords and DyNet traffic can then be read on the network.
- **Keeping the default password**, or choosing a weak one. Change all three login passwords during setup.
- **Physical access:** anyone who can press the system button, connect to USB or reach the RS-485 bus can change or use the device. Install it where only authorised people can reach it.
- **Not installing security updates.**

Secure setup

1. Install the device where only authorised people can reach it, and connect it to a private network.
2. Enter [configuration mode](#) and log in with the default password supplied with the device.
3. Under [Security](#), set new passwords for the web interface, WebSocket and TCP logins. Keep [Login required](#) and TLS on for every service unless a client can't support them.
4. Download the device CA certificate, check its fingerprint, and install it on each computer that connects - see [First connection and trusting the device](#).
5. Turn off [Web pages from other sites](#) (CORS) if no web app such as `dynet_ui` is used.
6. Install the latest firmware from the Releases page - see [Firmware](#).
7. [Save and reboot](#).

Changing a security setting (logins, TLS, CORS, the certificate) affects who can reach the device and whether its traffic is protected. The web page warns while any protection is off, and every change is recorded in the [system log](#).

Security updates

- **Support period:** security updates are provided free of charge for **7 years from the production date** of each unit. The end date (month and year) is on the label and box.
- The device can't update itself or tell you about updates, because it never connects to the internet. Check the [Releases](#) page, or use GitHub's **Watch** → **Custom** → **Releases** to be notified.
- Install updates from the web interface - see [Firmware](#). Security fixes are released as patch versions and marked in the changelog.
- Each release includes a software bill of materials ([SBOM.spdx](#)) and a vulnerability report ([SBOM_report.md](#)).

Removing data and taking the device out of service

Before selling, returning, passing on or disposing of the device, do a [Factory reset](#). It erases all settings, WiFi details, passwords, certificates, the USB drive, the crash report and the system log. In the EU, dispose of the device as electronic waste (WEEE), not with household waste.

© Copyright 2026 [Atkin Engineering](#). All Rights Reserved. [^copyright]

Specifications are subject to change without notice. No representation or warranty as to the accuracy or completeness of the information included herein is given and any liability for any action in reliance thereon is disclaimed.

[^usbpower]: USB Type-C port does not supply power.

[^rs485power]: RS-485 (parallel RJ12 6P6 ports) supplies power & data to *interfaceNode*.

[^jumpers]: Termination and duplex are both set via jumpers internal to the enclosure, not configurable from the web interface - changing either requires opening the unit. For duplex specifically: the jumper selects between the receiver always being enabled (full duplex) or tied to the transmit-enable line (half duplex). The firmware drives a single direction-control signal in either case - the jumper, not the firmware, determines which duplex mode results.

[^copyright]: © Copyright 2026 [Atkin Engineering](#). All Rights Reserved. ABN: 42 715 025 348.